SMIP-MWP
Sovereign Mohawk Internet Protocol
A Post-Quantum Secure, High-Performance Network Transport and Routing Protocol

rwilliamspbg-ops

May 2026

Contents

Abstract

The Sovereign Mohawk Internet Protocol (SMIP), also referred to as the Mohawk Wire Protocol (MWP), is a proposed next-generation network transport and routing protocol designed to address the dual challenges of post-quantum cryptographic security and high-performance packet forwarding. Derived from the original Sovereign Mohawk Protocol, a formally verified, Byzantine-resilient federated learning framework, SMIP-MWP extends these foundational guarantees into the network layer. This paper presents the architectural design, security model, performance characteristics, and formal verification strategy of SMIP-MWP. The protocol achieves deterministic hybrid post-quantum session cryptography with in-place authenticated encryption, kernel-bypass packet forwarding via AF_XDP with zero-copy semantics, and predictive

routing controls. Measured performance on development hardware demonstrates single-worker packet processing latency of 1,611 ns per operation (approximately 620,000 packets per second), with a best-tuned configuration achieving 1,487 ns per operation. The protocol targets sustained throughput of 10 Gbps or greater on appropriate hardware while maintaining sub-millisecond latency overhead and formal proofs of routing correctness.

1 Introduction

The accelerating development of quantum computing poses an existential threat to classical cryptographic protocols that underpin contemporary internet infrastructure. Transport Layer Security (TLS), IPsec, and similar protocols rely on the computational hardness of integer factorization and discrete logarithm problems — assumptions that quantum algorithms such as Shor's algorithm render invalid. Simultaneously, modern networking applications demand increasingly stringent performance characteristics: high-frequency trading platforms require microsecond-level latency, content delivery networks must sustain multi-gigabit throughput, and distributed systems require predictable routing convergence.

Existing solutions address either security or performance in isolation. Post-quantum VPN implementations typically sacrifice throughput for cryptographic assurance, while kernel-bypass networking frameworks prioritize speed over security guarantees. The Sovereign Mohawk Internet Protocol (SMIP-MWP) bridges this gap by integrating hybrid post-quantum cryptography with a high-performance kernel-bypass dataplane within a unified architectural framework backed by formal verification.

The contributions of this paper are threefold. First, we present the protocol architecture of SMIP-MWP, encompassing its wire format, hybrid post-quantum cryptographic session establishment, AF_XDP-based fast-path forwarding, and predictive routing engine. Second, we detail the security model, including replay protection, denial-of-service mitigation, and the hybrid cryptographic design combining ML-KEM key encapsulation with X25519 elliptic-curve Diffie-Hellman. Third, we report measured performance characteristics and outline the formal verification strategy using Lean 4.

2 Protocol Architecture

SMIP-MWP is organized as a layered protocol stack comprising four primary subsystems: the wire format layer, the hybrid cryptographic session layer, the AF_XDP fast-path dataplane, and the predictive routing engine. Each subsystem is designed for composability, testability, and independent optimization while maintaining coherent integration through well-defined interfaces.

2.1 Wire Format

The SMIP-MWP wire format defines a fixed 96-byte header structure designed for cache-line-friendly access patterns and efficient parsing. The header encodes packet type identifiers, session context, routing metadata, and integrity verification fields in a layout optimized for both software and potential hardware implementations.

The wire format specification supports multiple packet types including handshake initiation, handshake response, data transmission, routing update, and control messages. Each packet type carries type-specific payloads following the common header, enabling protocol extensibility without header format modification. Serialization and deserialization routines are implemented with zero-copy semantics where feasible, minimizing memory bandwidth consumption during high-throughput forwarding.

2.2 Hybrid Post-Quantum Cryptography

The cryptographic subsystem implements a hybrid key exchange mechanism combining Module Lattice-based Key Encapsulation Mechanism (ML-KEM, formerly Kyber) with X25519 elliptic-curve Diffie-Hellman. This dual-mechanism design provides cryptogenic diversity: even if either the lattice-based or elliptic-curve assumptions are compromised, the session remains secure. The design anticipates the Go 1.24 standard library inclusion of `crypto/mlkem`; until that availability, the Cloudflare CIRCL library provides interim post-quantum primitives.

Session encryption employs authenticated encryption with associated data (AEAD) using both AES-GCM and ChaCha20-Poly1305 cipher suites. The implementation supports in-place encryption and decryption operations, eliminating the need for separate plaintext and ciphertext buffers and reducing memory allocation pressure on the hot path. A deterministic nonce derivation scheme prevents nonce reuse vulnerabilities while avoiding the transmission of explicit nonce values.

Session management incorporates an LRU cache for established sessions, reducing the computational cost of repeated key lookups. Microbenchmark measurements demonstrate cached session creation at 545 ns per operation versus 610 ns for uncached creation, with in-place encryption completing in 833 ns per operation.

2.3 AF_XDP Fast-Path Dataplane

The fast-path dataplane utilizes the AF_XDP (Address Family eXpress Data Path) socket interface available in Linux kernels 5.10 and later. AF_XDP enables kernel-bypass networking where packets are delivered directly from the network interface card (NIC) driver to user-space memory without traversing the kernel networking stack, eliminating context switches, socket buffer copies, and protocol processing overhead.

The SMIP-MWP implementation employs zero-copy semantics through descriptor reuse and frame pooling. The FramePool type, built atop Go's `sync.Pool`, pre-allocates fixed-size buffers and recycles them after packet processing, reducing per-packet allocations to near zero. This design yields measured garbage collection pause times below 100 microseconds under sustained load, a critical requirement for latency-sensitive applications.

Multi-queue scaling is achieved through a linear architecture in which each NIC queue is serviced by a dedicated forwarding worker pinned to a specific CPU core. The `runtime.LockOSThread` mechanism prevents goroutine migration, preserving CPU cache affinity and eliminating cross-core synchronization. Initial measurements indicate that this architecture supports near-linear throughput scaling with additional queues.

2.4 Routing Engine

The routing engine implements a two-tier lookup mechanism comprising exact-match forwarding for established flows and longest-prefix-match (LPM) routing for destination-based forwarding. The exact-match path uses a sharded concurrent map with 16 shards to reduce mutex contention, providing approximately 16-fold reduction in lock contention compared to a single-lock design.

Predictive routing capabilities leverage federated learning models to anticipate topology changes and pre-compute alternative paths. While the predictive component remains under active development, the routing policy engine supports priority-ranked route selection and configurable fallback behaviors for unknown destinations.

3 Security Model

The SMIP-MWP security model addresses three primary threat vectors: passive eavesdropping, active replay attacks, and resource exhaustion denial-of-service attacks. The model assumes an adversary with network-level observation and injection capabilities but without access to endpoint private keys or legitimate session state.

3.1 Session Security

Each SMIP-MWP session derives unique encryption and authentication keys through the hybrid key exchange described in Section 2.2. The key derivation function produces separate keys for each direction of communication, ensuring that a compromise of one direction does not automatically compromise the reverse direction. Session keys are rotated according to configurable time and packet-count thresholds, with a default maximum session lifetime of 24 hours.

The in-place AEAD implementation authenticates both the packet payload and a subset of header fields, preventing tampering with routing metadata and packet type indicators. The associated data includes the session identifier, packet sequence number, and timestamp, binding the cryptographic integrity to the protocol context.

3.2 Replay Protection

Replay attack mitigation employs a 64-bit sliding window mechanism tracking recently received packet sequence numbers. Packets with sequence numbers falling below the window lower bound or duplicating previously observed values within the window are rejected before cryptographic verification, preventing resource consumption attacks that exploit repeated packet delivery.

Sequence number overflow detection identifies and handles the exhaustion of the 64-bit sequence number space through proactive session rekeying. When a sequence number approaches the maximum representable value, the protocol initiates a transparent session renegotiation, replacing the current session with a fresh key derivation before overflow can occur.

3.3 DoS Mitigation

Denial-of-service protection incorporates rate limiting at multiple protocol layers. The rate limiter restricts handshake initiation requests to prevent computational exhaustion through repeated key exchange operations, and per-session packet rate limiting identifies and isolates flows exhibiting anomalous traffic patterns.

The implementation includes memory-bound session state tables with automatic eviction of stale entries, preventing state exhaustion attacks that attempt to consume available memory through the creation of numerous incomplete session handshakes. Frame pooling further mitigates memory pressure by bounding the total buffer allocation regardless of incoming packet rate.

4 Performance Analysis

Performance evaluation follows a methodology combining synthetic microbenchmarks, sustained throughput tests, and profile-driven optimization. All benchmarks are conducted under controlled conditions with documented hardware configurations and reproducible test procedures.

4.1 Benchmark Methodology

The benchmark harness, implemented at `scripts/bench.sh`, executes Go's testing benchmark framework with configurable duration, profiling capture, and artifact retention. CPU and mem-

ory profiles are captured using pprof for post-run hotspot analysis. Synthetic benchmarks exercise individual components in isolation, while integration benchmarks evaluate end-to-end forwarding paths.

Hardware validation employs MoonGen, a high-speed packet generation framework based on DPDK, to produce sustained line-rate traffic for throughput measurement. Test configurations pin the packet generator and receiver processes to dedicated CPU cores, isolate NIC queues, and reserve hugepages to eliminate operating-system-level variability.

4.2 Results

Table 1 summarizes the primary benchmark results measured on a development AMD EPYC host under the synthetic AF_XDP harness with 60-second runs.

Table 1: Primary Performance Benchmarks (60-second runs)

Benchmark	**Latency (ns/op)**	**Memory (B/op)**	**Allocs/op**
AF_XDP Single-Worker (crypto)	2,014	560	9
AF_XDP 4-Worker (crypto)	2,376	632	11
AF_XDP Best-Tuned	1,487	424	7
Hybrid Session (cached)	545	1,392	4
Hybrid Session (uncached)	610	1,392	4
Encrypt In-Place	833	1,552	2

The canonical configuration for hardware validation uses `CRYPTO_WORKERS=1` and `CRYPTO_BATCH_SIZE=4` with pre-warmed HybridSession instances. Under this configuration, the best-tuned run achieved 1,487 ns per operation, corresponding to approximately 672,000 packets per second on a single worker thread.

Crypto microbenchmarks demonstrate efficient session establishment and packet encryption. The cached session creation path completes in 545 ns per operation with 1,392 bytes allocated per operation and 4 allocations. In-place encryption requires 833 ns per operation with 1,552 bytes and 2 allocations. These figures indicate that cryptographic operations constitute a manageable fraction of the overall per-packet processing budget.

5 Formal Verification

The SMIP-MWP formal verification strategy employs Lean 4 to establish machine-checkable proofs of critical protocol properties. The verification scope encompasses routing correctness, including loop-freedom and convergence bounds, as well as security invariants of the session state machine.

The Lean 4 formalization defines a mathematical model of the routing engine abstracting the concrete Go implementation into a state transition system. Properties are expressed as theorems over this model and proved using Lean's tactic framework. The loop-freedom theorem establishes that, assuming the routing update propagation protocol and the predictive routing fallback mechanism, no forwarding loop can persist indefinitely.

A conformance testing layer validates that the executable Go implementation respects the abstract Lean model through property-based testing. This dual approach — formal proof of the abstract model and conformance testing of the concrete implementation — provides assurance at both the specification and implementation levels.

6 Future Work

Several areas of active and planned development extend the current SMIP-MWP implementation toward full production deployment.

Complete handshake state machine implementation with timeout handling, retry backoff, and full ML-KEM integration remains the highest priority. The current implementation employs a partially stubbed key exchange pending Go 1.24 availability; the Cloudflare CIRCL library provides an interim path to full post-quantum operation.

AF_XDP descriptor reuse optimization will further reduce per-packet latency by pre-allocating descriptor slices and eliminating allocation in the poll loop. Dynamic batch sizing based on observed load patterns will improve amortization of TX submission overhead under variable traffic conditions.

The predictive routing subsystem requires integration with the federated learning infrastructure to enable intelligent path selection based on observed network conditions. Completion of the Lean 4 routing proofs and extension to cover the session state machine and cryptographic properties constitutes an ongoing research direction.

7 Conclusion

SMIP-MWP presents a unified approach to post-quantum secure, high-performance network communication. By integrating hybrid post-quantum key exchange, in-place authenticated encryption, kernel-bypass AF_XDP forwarding, and predictive routing within a formally verified architectural framework, the protocol addresses the dual challenges of cryptographic security and network performance that existing solutions approach separately.

Measured performance demonstrates the feasibility of the approach: single-worker packet processing at 1,487 ns per operation with cryptographic security active, and a linear scaling architecture supporting throughput aggregation across multiple NIC queues. The frame pooling and sharded session map optimizations achieve garbage collection pauses below 100 microseconds, satisfying the latency requirements of time-sensitive applications.

The formal verification strategy using Lean 4 establishes a foundation for machine-checkable correctness guarantees, with loop-freedom proofs and conformance testing providing assurance at both the specification and implementation levels. As the protocol matures through completion of the handshake state machine, hardware validation at 10 Gbps and beyond, and extension of formal proofs, SMIP-MWP aims to provide a production-ready transport and routing platform for the post-quantum networking era.

References

[1] Bernstein, D.J. and Lange, T., "Post-quantum cryptography", *Nature*, 549(7671), pp.188-194, 2017.

[2] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard", FIPS 203, 2024.

[3] Hoiland-Jorgensen, T., et al., "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel", ACM CoNEXT, 2018.

[4] Go Authors, "crypto/mlkem", Go Standard Library (Go 1.24+), https://pkg.go.dev/crypto/mlkem.

[5] Cloudflare, "CIRCL: Cloudflare Interoperable, Reusable Cryptographic Library", https://github.com/cloudflare/circl.

[6] de Moura, L. and Ullrich, S., "The Lean 4 Theorem Prover and Programming Language", CADE-28, 2021.

[7] NIST, "Post-Quantum Cryptography Standardization", https://csrc.nist.gov/projects/post-quantum-cryptography.

[8] Prometheus Authors, "Prometheus: Monitoring and Alerting", https://prometheus.io.

SMIP-MWP
https://github.com/rwilliamspbg-ops/SMIP-MWP